最小生成树

城镇修建

Z市是一个荒芜未经开发的新区划，里面的N个城镇间，村民们只能通过原本的M条狭窄的小路经行通行，十分的不方便。Z市政府决定，花钱请施工队，对其中的$N-1$条路进行重新修建，将小路修建成为大路，以确保**每个城镇**之间通行方便

但是Z市政府希望能尽可能省钱的方式完成施工，于是找到了你，请你为他们选择一条合适的修建方案，告诉他们修建的开销。

为了确保题目的多样性和丰富性，该问题设置了多个难度等级。

输入格式

输入的第一行有两个整数N和$M(1 \le N \le M \le 1e5)$，表示有N个城镇，M条初始的狭窄小路。

紧接着的M行分别是3个整数，u_i, v_i, w_i，表示城镇u_i和v_i间存在一条小路，将其修建成大路的开销是w_i

输出格式

输出一行，这一行只包含一个整数，表示本次修建的开销

做法

prime

```
#include<bits/stdc++.h>
using namespace std;

const int maxn = 2e5 + 10;
struct edge{
    int v, w;
    bool operator<(const edge &a)const{
        return w > a.w;
    }
};
vector<edge> E[maxn];
int vis[maxn], dis[maxn];
int main(){
    int n, m;cin>>n>>m;
    memset(vis, 0, sizeof(vis));
    memset(dis, 0x3f, sizeof(dis));
    for(int i = 0; i < m; i++){
        int u, v, w;
        cin >> u >> v >> w;
        E[u].push_back({v,w});
        E[v].push_back({u,w});
    }
    int ans = 0;
    priority_queue<edge,vector<edge>> q;
    q.push({1, 0});
    dis[1] = 0;
```

```
    int cnt = 0;
    while(!q.empty()){
        if (cnt >= n) break;
        int u = q.top().v;
        int d = q.top().w;
        q.pop();
        if (vis[u]) continue;
        vis[u] = 1;
        ++cnt;
        ans += d;
        for(auto e:E[u]){
            int v= e.v;
            int w = e.w;
            if (vis[v]) continue;
            if (w < dis[v]){
                dis[v] = w;
                q.push({v, dis[v]});
            }
        }
    }
    // cout << cnt << endl;
    cout << ans;
}
```

Kruskal

前提：并查集

```
#include<bits/stdc++.h>
using namespace std;

const int maxn = 2e5 + 10;
struct edge{
    int u, v, w;
    bool operator<(const edge &a)const{
        return w < a.w;
    }
}E[maxn];
int pa[maxn];
int find(int x){
    return pa[x] == x ? x : pa[x] = find(pa[x]);
}
void unite(int x, int y) {
    pa[find(x)] = find(y);
}
int main(){
    int n, m;cin>>n>>m;
    for(int i = 1; i <= n; i++) pa[i] = i;
    for(int i = 1; i <= m; i++){
        cin>>E[i].u>>E[i].v>>E[i].w;
    }
    sort(E + 1, E + m + 1);
    int ans = 0;
    int cnt = 0;
    for(int i = 1; i <= m; i++){
        int u = E[i].u, v = E[i].v, w = E[i].w;
        if(find(u) != find(v)){
```

```
            unite(u, v);
            ans += w;
            cnt++;
        }
    }
    // cout<<cnt<<endl;
    cout << ans;
}
```

难度二：最小生成树是否唯一

Z市政府对你设计的方案的开销他们很满意，但是对你设计的修建路线他们不是很满意，想要知道是否存在其他的方案使得仍然可以通过这样的开销完成修建。

输入格式

输入的第一行有两个整数N和$M(1 \le N \le M \le 1e5)$，表示有N个城镇，M条初始的狭窄小路。

紧接着的M行分别是3个整数，u_i, v_i, w_i，表示城镇u_i和v_i间存在一条小路，将其修建成大路的开销是w_i

输出格式

输出两行

第一行只包含一个整数，表示本次修建的开销

第二行输出最小生成树是否唯一，如果唯一，输出“unique”，否则输出“not unique”

做法

法一：暴力，每次删一个有效边接着跑最小生成树，然后比ans

法二：kruskal打个tag，prime同一时刻看值相同的有没有

```
#include<bits/stdc++.h>
using namespace std;

const int maxn = 2e5 + 10;
struct edge{
    int u, v, w;
    bool operator<(const edge &a)const{
        return w < a.w;
    }
}E[maxn];
int pa[maxn];
int find(int x){
    return pa[x] == x ? x : pa[x] = find(pa[x]);
}
void unite(int x, int y) {
    pa[find(x)] = find(y);
}
int main(){
    int n, m;cin>>n>>m;
    for(int i = 1; i <= n; i++) pa[i] = i;
    for(int i = 1; i <= m; i++){
        cin>>E[i].u>>E[i].v>>E[i].w;
```

```
    }
    sort(E + 1, E + m + 1);
    int ans = 0;
    int cnt = 0;
    int flag = 0;
    for(int i = 1; i <= m; i++){
        int u = E[i].u, v = E[i].v, w = E[i].w;
        int j = i;
        while(j<=m&&E[j].w==w)
            j++;
        int cnt1,cnt2;cnt1=cnt2=0;
        for(int k = i; k < j; k++){
            if(find(E[k].u)!=find(E[k].v)){
                cnt1++;
            }
        }
        for(int k = i; k < j; k++){
            if(find(E[k].u)!=find(E[k].v)){
                unite(E[k].u, E[k].v);
                ans += E[k].w;
                cnt2++;
                cnt++;
            }
        }
        if(cnt1>cnt2){
            flag = 1;
        }

    }
    // cout<<cnt<<endl;
    cout << ans << endl;
    if(flag)
        cout << "not unique";
    else
        cout << "unique";
}
```

难度三：次小生成树（严格）

因为得知Z市政府选择的是最便宜的方案，引起了居民们的热议。为了稳住居民们的内心，Z市政府决定不能再用任何开销等于最便宜开销的方案，但是同样他也不希望花太多的钱。因此，你需要为他设计出除了仅次于最便宜的开销，第二便宜的开销的方案，并输出此时的开销。

输入格式

输入的第一行有两个整数N和$M(1 \le N \le M \le 1e5)$，表示有N个城镇，M条初始的狭窄小路。

紧接着的M行分别是3个整数，u_i, v_i, w_i，表示城镇u_i和v_i间存在一条小路，将其修建成大路的开销是w_i

输出格式

输出一行，只包含一个整数，表示本次修建的开销

做法

法一：暴力，每次删一个有效边接着跑最小生成树，然后比ans

法二：kruskal+LCA倍增（或者树剖）:遍历所有的无效边(u,v,w)，加入mst后会多出一个环，找生成树上环路径最大的边w',减去，更新ans（但是为了确保是严格次小的，所以加入的边的值和最大的w'的值不能一样，LCA需要维护最大和次大的边）

```
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
const int maxn = 3e5 + 10;
const int inf = 0x3f3f3f3f;
const ll INF = 0x3f3f3f3f3f3f3f3f;

#define int long long
#define fast ios::sync_with_stdio(false),cin.tie(0),cout.tie(0)
struct edge{
    int u, v, w;
    bool operator<(const edge &a)const{
        return w < a.w;
    }
}E[maxn];
struct tree{
    int v, w;
};
int n, m;
vector<tree> T[maxn];
int pa[maxn];
int find(int x){
    return pa[x] == x ? x : pa[x] = find(pa[x]);
}
void unite(int x, int y) {
    pa[find(x)] = find(y);
}

int dep[maxn],f[maxn][21], sz[maxn], vx[maxn][21], vx1[maxn][21];
int vis[maxn];
void dfs_lca(int now,int fa,int val){
    dep[now] = dep[fa] + 1;
    f[now][0] = fa;
    vx[now][0] = val;
    vx1[now][0] = 0;
    for (int i = 1;(1<<i)<=dep[now];i++){
        f[now][i] = f[f[now][i-1]][i-1];
        vx[now][i] = max(vx[f[now][i - 1]][i - 1], vx[now][i - 1]);
        if(vx[now][i-1]==vx[f[now][i-1]][i-1]){
            vx1[now][i] = max(max(vx1[f[now][i - 1]][i - 1], vx1[now][i - 1]),
0ll);
        }
        else{
            vx1[now][i] = max(max(vx1[f[now][i - 1]][i - 1], vx1[now][i - 1]),
min(vx[now][i - 1], vx[f[now][i - 1]][i - 1]));
        }

    }
    for(auto i:T[now]){
        if(i.v != fa){
            dfs_lca(i.v, now, i.w);
```

```
        }
    }
}
int lca(int x,int y,int z){
    if(dep[x] < dep[y]) swap(x, y);
    int ans = 0,d=dep[x]-dep[y];
    for (int i = 0; (1 << i) <= d;i++){
        if(1<<i & d){
            ans = max(ans, (vx[x][i] == z ? vx1[x][i] : vx[x][i]));
            x = f[x][i];
        }
    }
    if(x==y)return ans;
    for(int i=log2(dep[x]);i>=0;i--){
        if(f[x][i] != f[y][i]){
            ans = max(max(vx[x][i], vx[y][i]) == z ? vx1[x][i] : max(vx[x][i],
vx[y][i]), ans);
            x = f[x][i];
            y = f[y][i];
        }
    }
    return (max(max(vx[x][0],vx[y][0])==z?max(max(vx1[x][0],vx1[y]
[0]),ans):max(vx[x][0],vx[y][0]),ans));
}
int ans = INF;
signed main(){
    fast;
    cin>>n>>m;
    for(int i = 1; i <= n; i++) pa[i] = i;
    for(int i = 1; i <= m; i++){
        cin>>E[i].u>>E[i].v>>E[i].w;
        if(E[i].u==E[i].v){
            i--;
            m--;
        }
    }
    sort(E + 1, E + m + 1);
    int anss = 0;
    int cnt = 0;
    for(int i = 1; i <= m; i++){
        int u = E[i].u, v = E[i].v, w = E[i].w;
        if(find(u) != find(v)){
            unite(u, v);
            anss += w;
            cnt++;
            vis[i] = 1;
            T[u].push_back({v, w});
            T[v].push_back({u, w});
        }
        if(cnt==n-1)
            break;
    }
    for (int i=1;i<=n;i++)
        if (!dep[i]) dfs_lca(i,0,0);
    for(int i = 1; i <= m; i++){
        if(!vis[i]){
            int x=lca(E[i].u,E[i].v,E[i].w);
            if(E[i].w-x!=0){
```

```
                ans=min(ans,E[i].w-x);
            }
        }
    }
    cout << anss + ans;
}
```